HW 4

Zhen Wu

Prob. 1

```
check_above <- function(x, y) {
  return(x > y)
}
```

(a) Function to check if a value is above a threshold

```
set.seed(42) # For consistent results
random_vec <- rnorm(n = 10, mean = 35, sd = 10)
loop_results <- logical(10)

for (i in 1:10) {
  loop_results[i] <- check_above(random_vec[i], 35)
}
print(random_vec)
```

(b) For loop using a random normal vector

```
##  [1] 48.70958 29.35302 38.63128 41.32863 39.04268 33.93875 50.11522 34.05341
##  [9] 55.18424 34.37286
```

```
print(loop_results)
```

```
##  [1]  TRUE FALSE  TRUE  TRUE  TRUE FALSE  TRUE FALSE  TRUE FALSE
```

```
# We use na.rm = TRUE because the airquality dataset contains NA values
ozone_check <- sapply(airquality$Ozone, check_above, y = 42)
ozone_above_42 <- sum(ozone_check, na.rm = TRUE)
cat("Number of days with Ozone > 42:", ozone_above_42, "\n")
```

(c) Using sapply on the airquality dataset

```
## Number of days with Ozone > 42: 44
```

```
target_data <- list(airquality$Ozone, airquality$Temp, airquality$Solar.R)
thresholds <- c(42, 77.88, 185.9)

multi_counts <- mapply(function(col_data, thresh) {
  sum(sapply(col_data, check_above, y = thresh), na.rm = TRUE)
}, target_data, thresholds)
```

```r
names(multi_counts) <- c("Ozone > 42", "Temp > 77.88", "Solar.R > 185.9")
print(multi_counts)
```

(d) Using mapply for multiple variables simultaneously

```
##      Ozone > 42    Temp > 77.88 Solar.R > 185.9
##              44              85              87
```

Prob. 2

(a) Fibonacci Implementation

Recursive approach

```r
fib_recursive <- function(n) {
  if (n <= 2) {
    return(1)
  } else {
    return(fib_recursive(n - 1) + fib_recursive(n - 2))
  }
}

# Iterative (loop) approach
fib_loop <- function(n) {
  if (n <= 2) return(1)
  fib_values <- numeric(n)
  fib_values[1:2] <- 1
  for (i in 3:n) {
    fib_values[i] <- fib_values[i - 1] + fib_values[i - 2]
  }
  return(fib_values[n])
}
```

```r
# Measuring execution time for both methods
time_recursive <- system.time(fib_recursive(30))
time_loop <- system.time(fib_loop(30))

cat("Time for Recursion (n=30):\n")
```

(b) Benchmarking for n = 30

```
## Time for Recursion (n=30):
```

```r
print(time_recursive)
```

```
##    user  system elapsed
##   0.183   0.003   0.185
```

```r
cat("\nTime for Loop (n=30):\n")
```

```
##
## Time for Loop (n=30):
```

```r
print(time_loop)
```

```
##    user  system elapsed
##   0.002   0.000   0.001
```

Prob. 3

The construction of the LU decomposition relies on the specific structure of the atomic lower triangular matrices $L_k = I - l_k e_k^T$. The first "stroke of luck" is found in the simplicity of the inverse of these matrices. Because the vector l_k has zeros in its first k entries, the inner product $e_k^T l_k$ is zero. This leads directly to the inverse:

$$L_k^{-1} = I + l_k e_k^T$$

The second stroke of luck involves the product of these inverses to form the final matrix L. When we multiply $L_1^{-1} L_2^{-1} \dots L_{m-1}^{-1}$, the "cross terms" of the form $l_i e_i^T l_j e_j^T$ vanish whenever $i < j$ because the i-th entry of l_j is zero. This allows the product to simplify to a basic sum of the rank-one updates:

$$L = \prod_{k=1}^{m-1} (I + l_k e_k^T) = I + \sum_{k=1}^{m-1} l_k e_k^T$$

The third stroke of luck is the culmination of the algorithm itself. By applying these Gauss transformations sequentially to the matrix A, we obtain an upper triangular matrix U. The combined effect of these algebraic properties ensures that we can write the original matrix as a product of a unit lower triangular matrix and an upper triangular matrix:

$$A = LU$$

Prob. 4

Let R be a nonsingular $m \times m$ upper-triangular matrix and define the subspace $S_n = \text{span}\{e_1, \dots, e_n\}$. Since R is upper-triangular, each column r_j satisfies $r_j \in S_j$. For any n, the first n columns span a subspace of S_n, and due to the nonsingularity of R, these columns are linearly independent. By a dimension argument, we have:

$$\text{span}\{r_1, \dots, r_n\} = S_n$$

To show that $X = R^{-1}$ is upper-triangular, we consider the equation $Rx_j = e_j$ for the j-th column of the inverse. Since $e_j \in S_j$, and we know that S_j is exactly the span of the first j columns of R, the vector e_j can be written as:

$$e_j = \sum_{i=1}^{j} c_i r_i = R \sum_{i=1}^{j} c_i e_i$$

Because R is nonsingular, the solution x_j is unique. The expression above implies that x_j must be a linear combination of only the first j standard basis vectors:

$$x_j = \begin{bmatrix} c_1 & \dots & c_j & 0 & \dots & 0 \end{bmatrix}^T$$

Since every column x_j has zeros below the j-th entry, the inverse matrix R^{-1} is upper-triangular.

Prob. 5

For the induced 2-norm, we evaluate the maximum magnification of a unit vector x. Using the definition of the matrix norm and the property that v^*x is a scalar, we have:

$$\|E\|_2 = \sup_{\|x\|_2=1} \|uv^*x\|_2 = \|u\|_2 \sup_{\|x\|_2=1} |v^*x|$$

By applying the Cauchy-Schwarz inequality, the maximum value of $|v^*x|$ for a unit vector x is exactly $\|v\|_2$. Thus, the 2-norm of the outer product is the product of the Euclidean norms of the vectors:

$$\|E\|_2 = \|u\|_2\|v\|_2$$

Regarding the Frobenius norm, we use the trace identity $\|E\|_F^2 = \mathrm{Tr}(E^*E)$. Substituting $E = uv^*$ and using the cyclic property of the trace, we obtain:

$$\|E\|_F^2 = \mathrm{Tr}(vu^*uv^*) = \mathrm{Tr}(u^*uv^*v)$$

Since $u^*u = \|u\|_2^2$ and $v^*v = \|v\|_2^2$ are scalars, the trace of their product is simply the product itself:

$$\|E\|_F^2 = \|u\|_2^2\|v\|_2^2$$

Taking the square root proves that $\|E\|_F = \|u\|_2\|v\|_2$. Therefore, the relationship holds true for both the induced 2-norm and the Frobenius norm in the case of a rank-one matrix.